

MAIE 5532: Machine Learning Systems

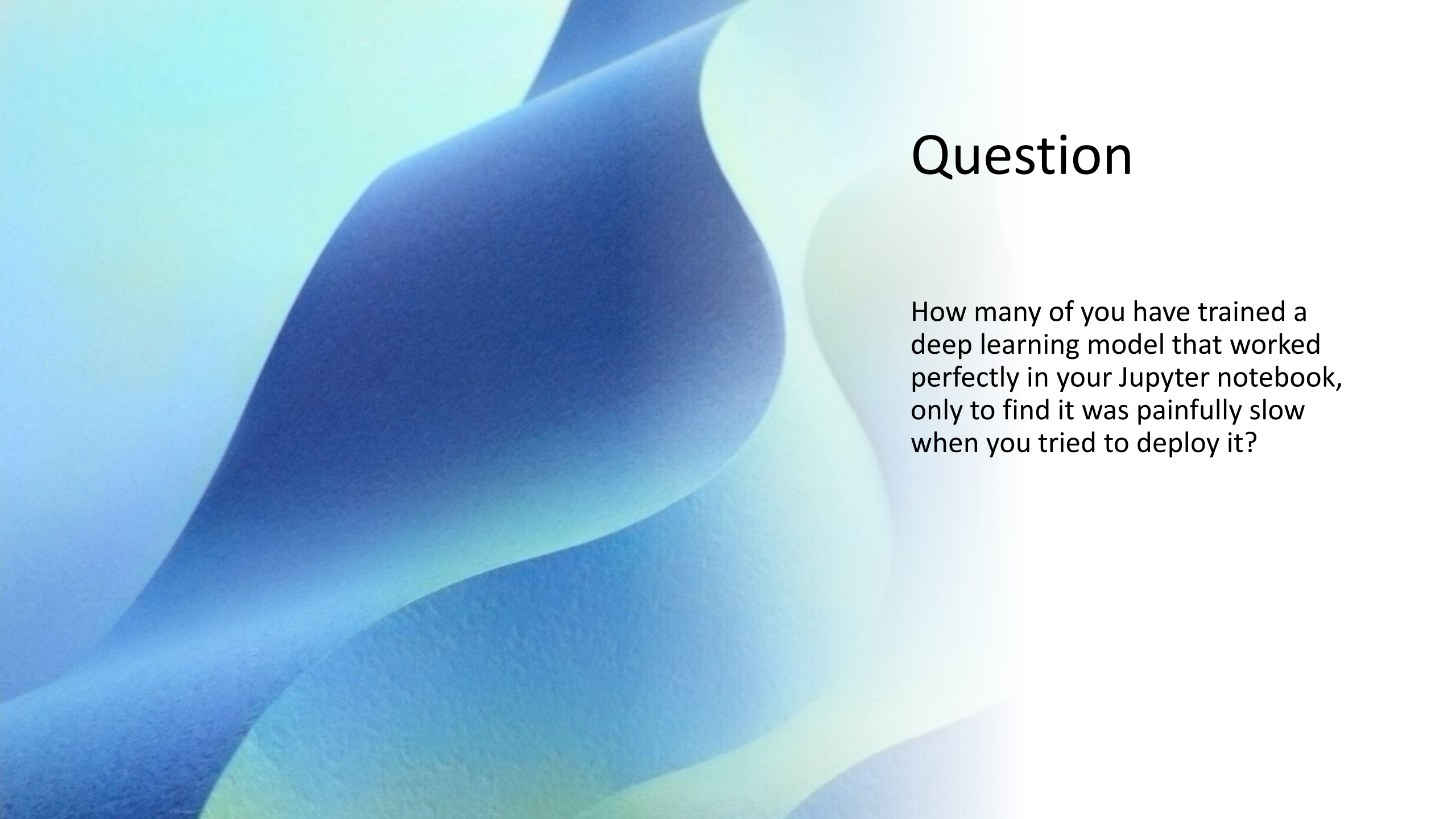
Week 3 - Graph-level Optimizations

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology



Question

How many of you have trained a deep learning model that worked perfectly in your Jupyter notebook, only to find it was painfully slow when you tried to deploy it?

The computation graphs that our ML frameworks create, are often far from optimal for actual deployment

Like building a house inefficiently



Week 3 Overview

Graph-level Optimizations: Transforming ML Computations for Efficiency

Learning Objectives:

1. Master computation graph optimization techniques and transformations
2. Design efficient model conversion and optimization pipelines
3. Evaluate latency optimization strategies across deployment targets

Required Readings:

- Book 1 – Volume 1 : ML Frameworks
- Book 1 – Volume 1 : Hardware Acceleration
- Book 1 – Volume 1 : Model Serving
- Book 2: Chapter 19: Porting Models from TensorFlow to TensorFlow Lite
- Book 2: Chapter 15: Optimizing Latency

Session Structure

Today's Organization

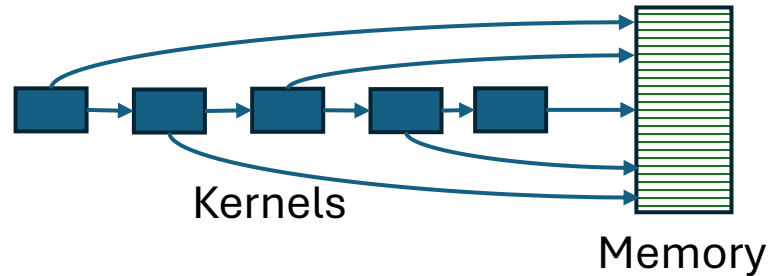
- **Session 1: Computation Graph Optimization**
 - Graph transformation techniques
 - Kernel fusion strategies
 - Redundant computation elimination
 - Memory-aware dataflow optimization
- **Session 2: Model Conversion and Compilation**
 - TensorFlow to TensorFlow Lite conversion pipeline
 - Automated optimization for deployment targets
 - Cross-platform compilation challenges
 - Operation coverage and compatibility
- **Session 3: Latency Optimization Strategies**
 - Performance profiling and bottleneck identification
 - Hardware-specific optimizations
 - Code optimization techniques
 - Real-time optimization approaches

The Optimization Challenge

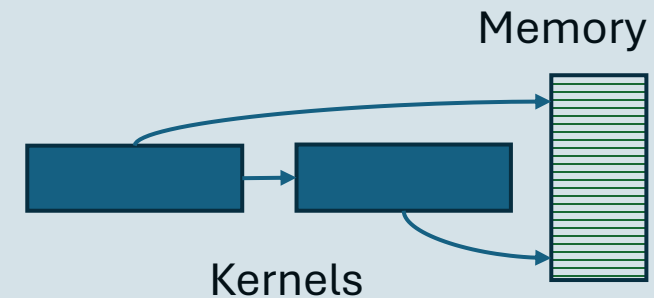
ML runs as computation graphs (ops + dataflow)

Naive execution wastes time:

- redundant ops
- extra memory traffic
- many small kernel launches
- poor hardware utilization



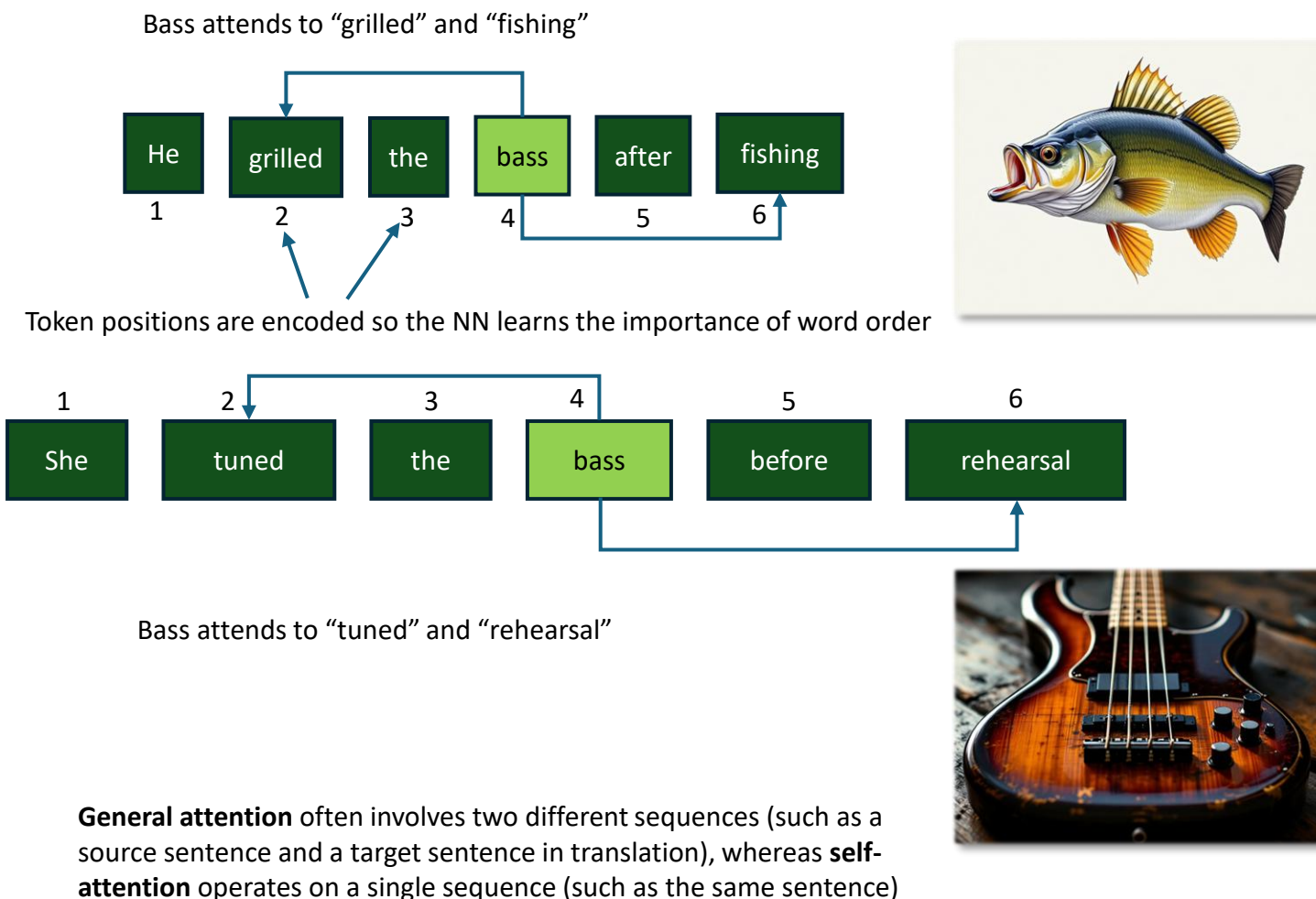
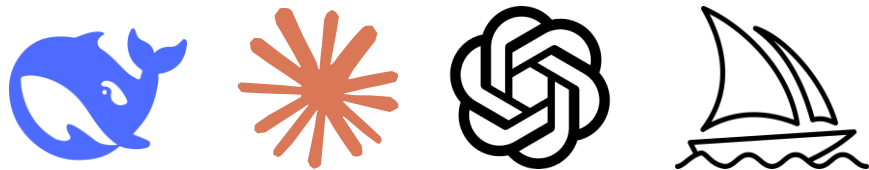
Goal: fuse, reorder, and reuse to cut I/O and launches



What Are Transformers? Where Are They Used?

- A neural-network architecture for sequences
- Core ideas: self-attention + positional encoding
- Parallel over tokens (no recurrence), scales well
- Learns context: meaning depends on neighbors
- Powers many modalities: text, vision, audio, code
- Backbone of models such as DeepSeek, Claude, GPT, Midjourney.

GPT: Generative Pre-trained Transformer



Transformers: Example & why we need optimizations

1) Input matrix

Input sentence (sequence): “The cat sat” (*sequence_length=3*)

1) The sentence is broken into tokens, each of them represented by an embedding vector

“the”: [0.2, -0.5, 0.7], “cat”: [-0.1, 0.3, 0.9], “sat”: [-0.2, 0.8, -0.3]

Input matrix: $X = \begin{bmatrix} [0.2 & -0.5 & 0.7] \\ [-0.1 & 0.3 & 0.9] \\ [-0.2 & 0.8 & -0.3] \end{bmatrix}$

Token: a piece of text (here: “the”, “cat”, “sat”).

Embedding: a list of numbers that represents a token’s meaning. Embeddings are learned during training.

We use small 3×3 matrices for teaching. Real models use far larger sizes (hundreds to thousands of numbers per token), but the steps are identical.



When we input the sentence "the cat sat" to a language model, the primary goal is **prediction**. Specifically, **we want the model to predict the most likely next word (or words) in the sequence.**

Transformers: Example & why we need optimizations

2) W_q, W_k, W_v Weight Matrices



Input sentence: “The cat sat”

2) We **randomly** create weight matrices for **Query** (W_q), **Key** (W_k), and **Value** (W_v):

- They are learned weight matrices that project the same input vector into three different roles: the Query searches for relevant information, the Key indicates what information it contains, and the Value is the actual information to be retrieved.
- In this example, the size is 3x3

Query weight matrix

$$(W_q) = \begin{bmatrix} [0.1 & -0.8 & 0.3] \\ [-0.2 & 0.5 & 0.6] \\ [-0.1 & 0.9 & -0.2] \end{bmatrix}$$

Key weight matrix

$$(W_k) = \begin{bmatrix} [-0.8 & -0.3 & 0.1] \\ [-0.3 & 0.4 & 0.5] \\ [-0.2 & 0.3 & -0.7] \end{bmatrix}$$

Value weight matrix

$$(W_v) = \begin{bmatrix} [-0.1 & -0.3 & 0.2] \\ [0.1 & 0.4 & 0.4] \\ [-0.1 & 0.1 & -0.2] \end{bmatrix}$$

Transformers: Example & why we need optimizations

3) Q, K, V Matrices



Input sentence: “The cat sat”

3) We compute the **Query, Key, and Value Matrices**

- **Via a “Linear transformation”** -> takes the original word embeddings and projects them into a “Query Space”

$$\begin{array}{ccc} \text{Input Matrix } X & \text{Query weight matrix } W_q & \text{Query Matrix } Q \\ \begin{bmatrix} 0.2 & -0.5 & 0.7 \\ -0.1 & 0.3 & 0.9 \\ -0.2 & 0.8 & -0.3 \end{bmatrix} \times \begin{bmatrix} 0.1 & -0.8 & 0.3 \\ -0.2 & 0.5 & 0.6 \\ -0.1 & 0.9 & -0.2 \end{bmatrix} & = & \begin{bmatrix} 0.19 & 1.13 & -0.62 \\ -0.20 & 0.74 & 0.05 \\ -0.35 & 0.11 & 0.77 \end{bmatrix} = Q \end{array}$$

Transforms words into "questions"

$$\begin{array}{ccc} \text{Input Matrix } X & \text{Key weight matrix } W_k & \text{Key Matrix } K \\ \begin{bmatrix} 0.2 & -0.5 & 0.7 \\ -0.1 & 0.3 & 0.9 \\ -0.2 & 0.8 & -0.3 \end{bmatrix} \times \begin{bmatrix} 0.1 & -0.8 & 0.3 \\ -0.2 & 0.5 & 0.6 \\ -0.1 & 0.9 & -0.2 \end{bmatrix} & = & \begin{bmatrix} 0.05 & -0.52 & -0.18 \\ -0.17 & 0.37 & 0.26 \\ -0.02 & 0.15 & 0.34 \end{bmatrix} = K \end{array}$$

Transforms words into "answers"

$$\begin{array}{ccc} \text{Input Matrix } X & \text{Value weight matrix } W_v & \text{Value Matrix } V \\ \begin{bmatrix} 0.2 & -0.5 & 0.7 \\ -0.1 & 0.3 & 0.9 \\ -0.2 & 0.8 & -0.3 \end{bmatrix} \times \begin{bmatrix} 0.1 & -0.8 & 0.3 \\ -0.2 & 0.5 & 0.6 \\ -0.1 & 0.9 & -0.2 \end{bmatrix} & = & \begin{bmatrix} -0.19 & -0.01 & 0.34 \\ 0.07 & 0.46 & 0.29 \\ 0.06 & 0.17 & -0.30 \end{bmatrix} = V \end{array}$$

Transforms words into "content"

Transformers: Example & why we need optimizations

4) Attention Scores and Weights



Input sentence: “The cat sat”

4) We compute **Attention Scores** and **Attention Weights**

Attention Scores: $Q \cdot K^T$

$$\begin{bmatrix} [0.19 & 1.13 & -0.62] \\ [-0.20 & 0.74 & 0.05] \\ [-0.35 & 0.11 & 0.77] \end{bmatrix} \cdot \begin{bmatrix} [0.05 & -0.52 & -0.18] \\ [-0.17 & 0.37 & 0.26] \\ [-0.02 & 0.15 & 0.34] \end{bmatrix}^T = \begin{bmatrix} [0.274 & -0.468 & -0.098] \\ [-0.079 & 0.112 & -0.074] \\ [-0.002 & -0.015 & -0.268] \end{bmatrix}$$

We Compare (by dot product) each token's Query to each token's Key

Attention Weights: $\text{softmax}(\text{Attention Scores} / \sqrt{d_K})$

$$\text{softmax}\left(\begin{bmatrix} [0.274 & -0.468 & -0.098] \\ [-0.079 & 0.112 & -0.074] \\ [-0.002 & -0.015 & -0.268] \end{bmatrix} / \sqrt{3}\right) = \begin{bmatrix} [0.417 & 0.275 & 0.308] \\ [0.328 & 0.334 & 0.338] \\ [0.388 & 0.385 & 0.226] \end{bmatrix}$$

- We scale by $\sqrt{d_K}$ so numbers don't get too large as dimensions grow
- Softmax turns each row into a set of weights that sum to 1

Transformers: Example & why we need optimizations

5) Weighted Sum

Input sentence: “The cat sat”

5) We Mix Values to produce outputs



Outputs = Attention Weights · V

$$\begin{bmatrix} 0.417 & 0.275 & 0.308 \\ 0.328 & 0.334 & 0.338 \\ 0.388 & 0.385 & 0.226 \end{bmatrix} \cdot \begin{bmatrix} -0.19 & -0.01 & 0.34 \\ 0.07 & 0.46 & 0.29 \\ 0.06 & 0.17 & -0.30 \end{bmatrix} = \begin{bmatrix} 0.010 & 0.205 & 0.109 \\ -0.022 & 0.213 & 0.111 \\ -0.034 & 0.201 & 0.083 \end{bmatrix}$$

- Each row of Attention Weights is a “recipe” for mixing the Values V
- The output O is a context-aware representation
- In deeper layers, this process repeats



Transformers: What are heads? Why do they cause extra memory traffic?



- **One “head”** = one copy of the attention steps you just saw, with its own W_q , W_k , W_v .
- **Multi-head attention** = several heads run in parallel; each head looks for different patterns.
- All heads read the same sequence’s Keys and Values for their calculations.

Tiny example intuition:

- If we had $H=4$ heads for “the cat sat,” each head would:
 - Build its own Q , K , V
 - Compute scores and mix V
 - Naively, each head repeatedly reloads K and V from memory for each token → many repeated reads.
- 

Multiple Attention Heads

Let's say we have n attention heads with different Weight Matrices

$$W_{q_1}, W_{k_1}, W_{v_1}, W_{q_2}, W_{k_2}, W_{v_2}, \dots, W_{q_n}, W_{k_n}, W_{v_n}$$





Multiple Attention Heads

Memory accesses of the input matrix X we need to compute the Query matrix Q for all attention heads

Compute Q_1

Access $X[0]$ -> Multiply with W_{q_1} -> Store $Q_1[0]$

Access $X[1]$ -> Multiply with W_{q_1} -> Store $Q_1[1]$

...

Access $X[d_X]$ -> Multiply with W_{q_1} -> Store $Q_1[d_X]$

Compute Q_2

Access $X[0]$ -> Multiply with W_{q_2} -> Store $Q_2[0]$

Access $X[1]$ -> Multiply with W_{q_2} -> Store $Q_2[1]$

...

Access $X[d_X]$ -> Multiply with W_{q_2} -> Store $Q_2[d_X]$

...

Compute Q_n

..... Many X (input) memory accesses.....

Multiple Attention Heads

Memory accesses of the input matrix X we need to compute the Key matrix K for all attention heads

Compute K_1

Access $X[0]$ -> Multiply with W_{k_1} -> Store $K_1[0]$

Access $X[1]$ -> Multiply with W_{k_1} -> Store $K_1[1]$

...

Access $X[d_X]$ -> Multiply with W_{k_1} -> Store $K_1[d_X]$

Compute K_2

Access $X[0]$ -> Multiply with W_{k_2} -> Store $K_2[0]$

Access $X[1]$ -> Multiply with W_{k_2} -> Store $K_2[1]$

...

Access $X[d_X]$ -> Multiply with W_{k_2} -> Store $K_2[d_X]$

...

Compute K_n

..... Many X (input) memory accesses.....





Multiple Attention Heads

Memory accesses of the input matrix X we need to compute the Value matrix V for all attention heads

Compute V_1

Access $X[0]$ -> Multiply with W_{v_1} -> Store $V_1[0]$

Access $X[1]$ -> Multiply with W_{v_1} -> Store $V_1[1]$

...

Access $X[d_X]$ -> Multiply with W_{v_1} -> Store $V_1[d_X]$

Compute V_2

Access $X[0]$ -> Multiply with W_{v_2} -> Store $V_2[0]$

Access $X[1]$ -> Multiply with W_{v_2} -> Store $V_2[1]$

...

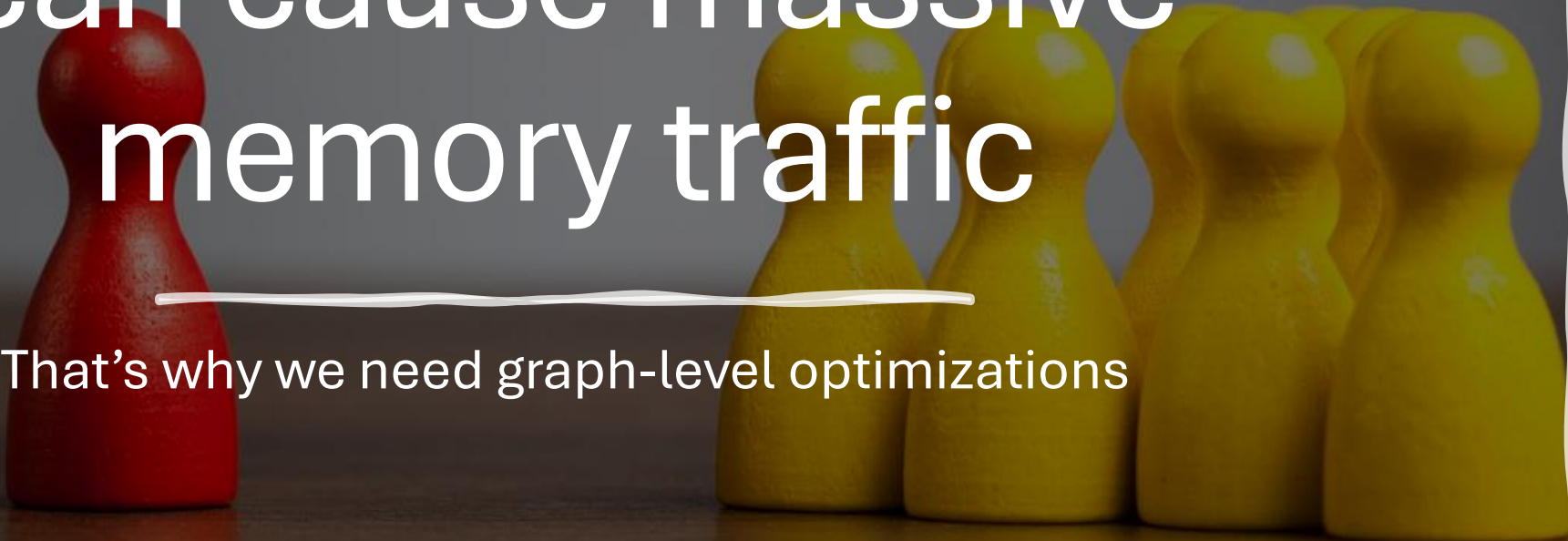
Access $X[d_X]$ -> Multiply with W_{v_2} -> Store $V_2[d_X]$

...

Compute V_n

..... Many X (input) memory accesses.....

Models like Transformers can cause massive memory traffic

A red chess piece, likely a king or queen, stands on the left. To its right is a row of five yellow chess pieces, likely pawns, standing in a line. They are on a dark wooden surface. The background is a dark, out-of-focus grey.

That's why we need graph-level optimizations

Multiple Attention Heads

Instead of Computing Q_i, K_i, V_i Separately

Memory accesses of the input matrix X we need to compute the **Value matrix V** for all attention heads

Compute V_1

Access $X[0]$ -> Multiply with W_{v_1} -> Store $V_1[0]$

Access $X[1]$ -> Multiply with W_{v_1} -> Store $V_1[1]$

...

Access $X[d_X]$ -> Multiply with W_{v_1} -> Store $V_1[d_X]$

Compute V_2

Access $X[0]$ -> Multiply with W_{v_2} -> Store $V_2[0]$

Access $X[1]$ -> Multiply with W_{v_2} -> Store $V_2[1]$

...

Access $X[d_X]$ -> Multiply with W_{v_2} -> Store $V_2[d_X]$

...

Compute V_n

.. Many X (input) memory accesses...

Multiple Attention Heads (Optimized)

Fuse kernels and process each input vector once

for i in range $(0, sequence_length)$

$x_i = X[i]$ # Access once

$V_1[i] = x_i @ W_{v_1}$

$V_2[i] = x_i @ W_{v_2}$

...

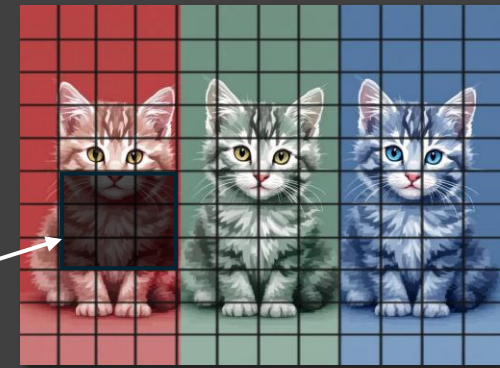
$V_n[i] = x_i @ W_{v_n}$

Less memory accesses



CNNs refresher

Filter



What goes in

- Input: 3D tensor (H, W, C). Example: 224×224×3 (RGB); grayscale: C=1.

How it works

- Filters/Kernels: small matrices (e.g., 3×3) with depth C; each filter → 1 output channel.
- Convolution: slide, dot-product → feature score per location.
- Output feature maps: stack of channels (e.g., 64) showing where patterns appear.

Shapes

- One filter on H×W×C → H_{out}×W_{out}×1
- N filters → H_{out}×W_{out}×N
- H_{out}/W_{out} = floor((Input−Kernel)/Stride) + 1; use Padding “same” to keep size.

Stride & Padding

- Stride 1: near-same size; Stride 2: halves H,W.
- Padding: “valid” shrinks; “same” preserves.

BatchNorm + ReLU

- BN: normalize each channel; ReLU: max(0,x) adds non-linearity.

BatchNorm Example

Batch: Input of 2 samples, 3 features each

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

mean μ and variance σ for each feature:

$$\mu = [2.5 \quad 3.5 \quad 4.5]$$

$$\sigma = [2.25 \quad 2.25 \quad 2.25]$$

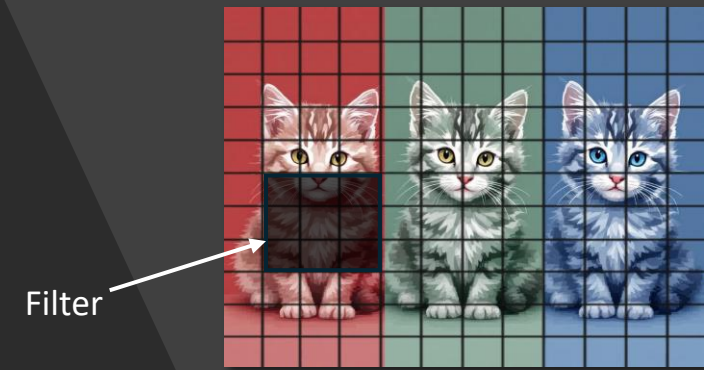
Normalize the batch:

$$\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \varepsilon}, \text{ (where: } \varepsilon = 0.001 \text{)}$$

Normalized batch

$$\begin{bmatrix} -0.9998 & -0.9998 & -0.9998 \\ 0.9998 & 0.9998 & 0.9998 \end{bmatrix}$$

CNNs – Fusion Opportunity



Naive Execution Problems:

R: Read from Memory

W: Write to Memory

Input $\rightarrow$ R Conv2D $W \rightarrow$ R BatchNorm $W \rightarrow$ R ReLU $W \rightarrow$ Output

Each operation writes intermediate results to memory

- Separate kernel launches for each operation
- Poor data locality
- Underutilized hardware resources

Optimization Opportunity:

Input $\rightarrow$ R [Conv2D + BatchNorm + ReLU] $W \rightarrow$ Output

- Fused operations eliminate intermediate memory writes and reduce kernel launches.
- It is like taking a direct highway instead of stopping at every traffic light!

Kernel Fusion Techniques

Merging Operations for Efficiency

Common Fusion Patterns:

1. Conv-BatchNorm-Activation

- $conv_output = conv2d(input, weights) + bias$
- $norm_output = batch_norm(conv_output, scale, offset)$
- $final_output = relu(norm_output)$
- Each step writes to memory and reads from memory

After fusion, the above operations become a single (fused) kernel with no intermediate memory writes.

2. Element-wise Operation Fusion

Unfused:

Fused

- $result[i] = relu((x[i] + y[i]) \times z[i])$

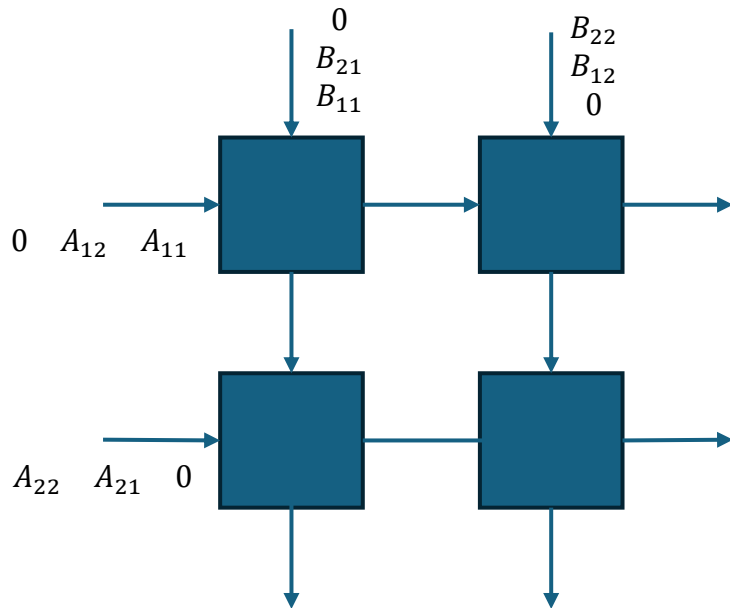
Memory Access Pattern Optimization

Optimizing Data Movement

- **Tiling for Systolic Arrays**
- **Data Layout Transformations**
- **Cache-aware Scheduling**

Tiling for Systolic Arrays

- A **systolic array** is a parallel computing hardware structure of connected processing elements (cells) that rhythmically process and transmit data
- TPUs** use systolic arrays as their core computational component to efficiently perform the high-throughput matrix multiplication required for AI workloads



Systolic 2x2 multiplication example: A tiles shift to the right and B tiles shift down. Each processing element multiplies its local A and B values and adds to a running sum. After several beats, the correct outputs emerge at the bottom/right without ever stopping the flow.

A "tile" is a pre-defined, fixed-size chunk of a larger matrix.

$$A \times B = C$$

A_{11}	A_{12}	A_{13}	0
A_{21}	A_{22}	A_{23}	0
A_{31}	A_{32}	A_{33}	0
0	0	0	0

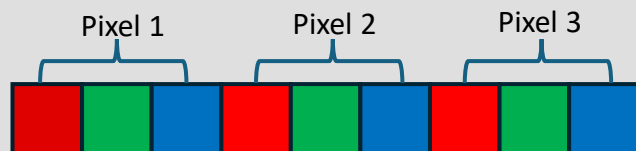
B_{11}	B_{12}	B_{13}	0
B_{21}	B_{22}	B_{23}	0
B_{31}	B_{32}	B_{33}	0
0	0	0	0

Both Matrices A and B are partitioned into fixed-size tiles that match the array's row bandwidth. Zeros pad the boundary, so every tile is the same size and enters on the same rhythm. As each A tile streams to the right and each B tile streams to the bottom, its values are reused across multiple columns of compute cells, letting one load of A and B participate in many multiplications before it exits.

Data Layout Transformations

- N: Number of images in the batch.
- C: Number of Channels per image (e.g., 3 for Red, Green, Blue).
- H: Height of the image in pixels.
- W: Width of the image in pixels.

NHWC



- **Order:** Batch, Height, Width, Channels
- **What it means:** For each pixel, all its color/feature values are stored next to each other.
- **Typical:** TPUs/mobile NPUs; kernels that combine channels at a pixel
- Order in the above image: pixel 1 (red) → pixel 1 (green), pixel 1 (blue), pixel 2 (red), pixel

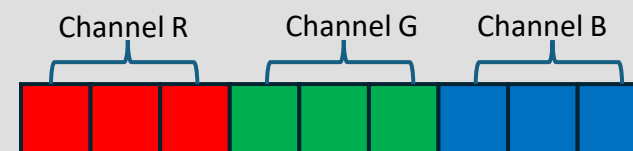
Reorder/transpose axes



Example permutation: [0, 3, 1, 2]
turns NHWC into NCHW

*No data changes—just the order
we store it.*

NCHW



- **Order:** Batch, Channels, Height, Width
- **What it means:** For each channel, all the pixels in the image are stored next to each other.
- **Good when:** You sweep over images one channel at a time.
- **Typical:** Convolution on many GPUs

For large matrix multiplies, we often go beyond NHWC/NCHW and use packed tile layouts that match the GEMM kernel

GEMM stands for General Matrix Multiplication. It's the mathematical name for the core operation that powers deep learning and scientific computing: multiplying two large matrices together.

Cache-aware Scheduling

Cache is a small, fast memory layer that sits between the CPU and main memory to reduce data access latency. It stores recently used (and nearby) data so the processor can reuse it quickly. L1 cache is the smallest and fastest, located closest to the CPU cores and split into instruction/data caches. L2 is larger but a bit slower, often private per core. L3 is much larger and slower than L1/L2, typically shared across cores on a chip, helping reduce trips to main memory by serving common data.

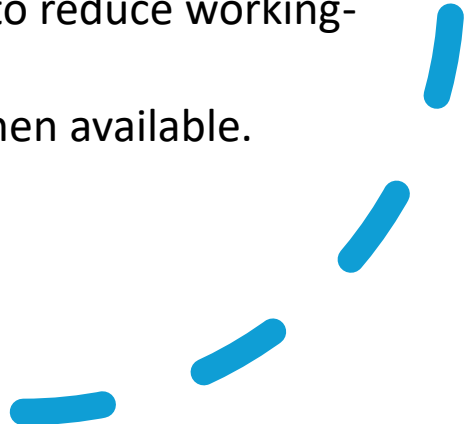
Why it matters:

- Minimizes cache misses by keeping hot data in faster caches (L1/L2) longer.

How it works:

- Reorder and group operations to reuse data while it's still in cache.
- Tile/block loops so working sets fit in L1/L2; promote shared data to L3-friendly batches.
- Align access patterns for spatial and temporal locality.

Practical tactics:

- Loop tiling/blocking, fusion, and interchange to reduce working-set size.
 - Prefetching and software-managed buffers when available.
- 



Computation Reordering

*Do expensive work less often, and
when you do it, make it count.*

```
# Inefficient: Recompute key-value pairs for each head
for head in attention_heads:
    keys = linear_transform(input, W_k[head])
    values = linear_transform(input, W_v[head])
    attention_output[head] = attention(queries[head], keys, values)

# Optimized: Compute all keys and values once
all_keys = linear_transform(input, W_k_combined)
all_values = linear_transform(input, W_v_combined)
for head in attention_heads:
    attention_output[head] = attention(queries[head],
                                      all_keys[head], all_values[head])
```

Parallelization Opportunities:

Identify operations that can execute concurrently without data dependencies.



Redundant Computation Elimination

Identifying and Removing Unnecessary Work

Common Redundancy Patterns:

1. Duplicate Subexpressions:

```
# Redundant
loss1 = mse(predictions, targets)
loss2 = mse(predictions, targets) # Same computation

# Optimized
shared_loss = mse(predictions, targets)
loss1 = shared_loss
loss2 = shared_loss
```

2. Residual Connection Optimization:

```
# In ResNet blocks, common subexpressions appear
def residual_block(x):
    conv1_out = conv2d(x, weights1)
    conv2_out = conv2d(conv1_out, weights2)
    return x + conv2_out # x is reused
```

3. Constant Folding:

```
# Constants computed at compile time
scaling_factor = 1.0 / 255.0 # Computed once, not per
inference
normalized = input * scaling_factor
```

Identifying and Removing Unnecessary Work



Advanced Graph Transformations

Sophisticated Optimization Techniques

1. Loop Unrolling for RNNs:

```
# Original: Dynamic loop
for t in range(sequence_length):
    hidden[t] = rnn_cell(input[t], hidden[t-1])

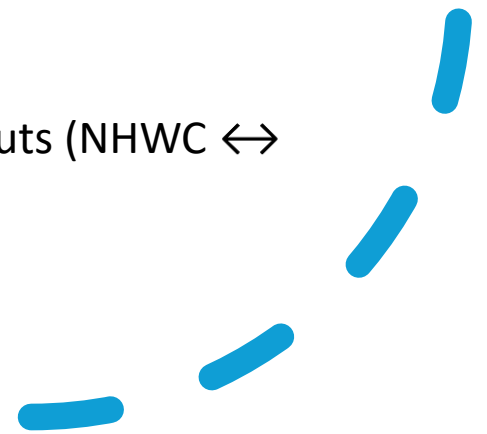
# Optimized: Unrolled for fixed sequence lengths
hidden[0] = rnn_cell(input[0], initial_hidden)
hidden[1] = rnn_cell(input[1], hidden[0])
hidden[2] = rnn_cell(input[2], hidden[1])
# ... enables further optimizations
```

2. Algebraic Simplifications:

```
# Matrix multiplication reordering
#  $A \times (B \times C)$  vs  $(A \times B) \times C$ 
# Choose order based on dimensions to minimize operations
```

3. Memory Layout Transformations:

Automatic conversion between different tensor layouts (NHWC ↔ NCHW) based on hardware preferences



Compiler-based Graph Optimization

Automated Transformation Systems

Pattern Recognition and Rewrite Rules:

```
# Compiler identifies patterns and applies transformations
pattern = "Conv2D → BatchNorm → ReLU"
replacement = "FusedConvBnRelu"

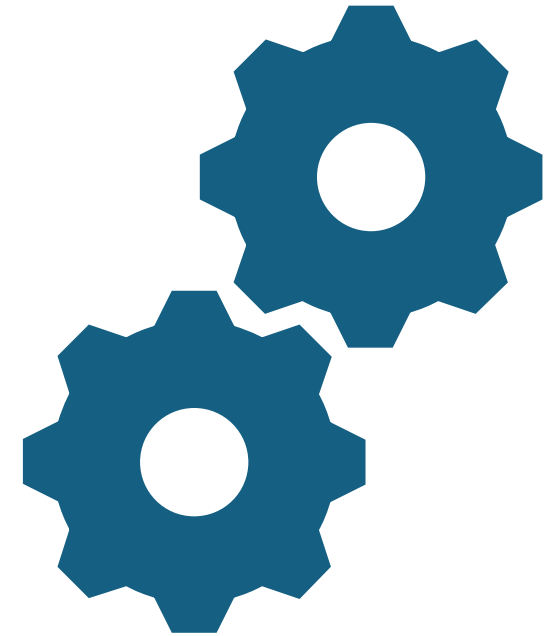
# Systematic application across entire graph
for subgraph in computation_graph:
    if matches_pattern(subgraph, pattern):
        replace_with(subgraph, replacement)
```

Multi-stage Optimization:

1. **High-level transformations:** Operation fusion, redundancy elimination
2. **Hardware-specific optimizations:** Layout transformations, kernel selection
3. **Low-level optimizations:** Instruction scheduling, register allocation

Optimization Verification:

Ensure mathematical equivalence through formal verification or numerical testing.



Graph Optimization Impact

Performance and Efficiency Gains

Bottleneck Mitigation:

- **Memory stalls:** Reduced through better data locality
- **Kernel launch overhead:** Minimized via operation fusion
- **Cache misses:** Improved through access pattern optimization

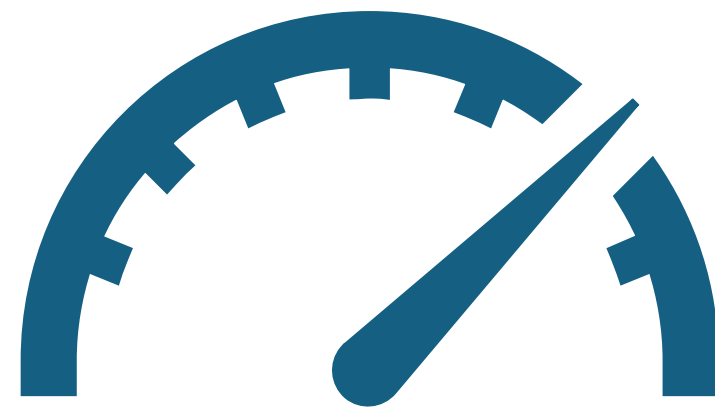
Hardware Utilization:

- **TPUs:** Optimized matrix multiplication tiling
- **GPUs:** Coalesced memory access patterns
- **CPUs:** Vectorization-friendly operations

Real-world Example:

Large Transformer model on edge device:

- **Before optimization:** 2.5 seconds inference, high memory bandwidth
- **After optimization:** 0.8 seconds inference, 60% reduction in memory traffic
- **Enabled real-time operation** on resource-constrained hardware



Exercise 1

(10 minutes)

[Download the exercise](#)

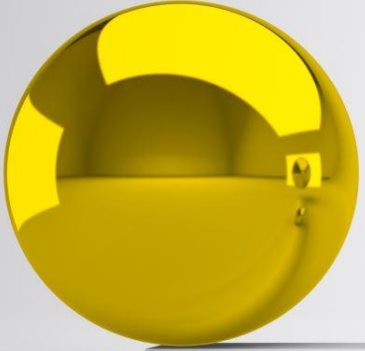
Graph Optimization Analysis

Scenario: CNN Layer Sequence

```
def inefficient_cnn_block(input_tensor):  
    # Layer 1  
    conv1 = conv2d(input_tensor, weights1, bias1)  
    norm1 = batch_norm(conv1, scale1, offset1)  
    act1 = relu(norm1)  
  
    # Layer 2  
    conv2 = conv2d(act1, weights2, bias2)  
    norm2 = batch_norm(conv2, scale2, offset2)  
    act2 = relu(norm2)  
  
    # Element-wise operations  
    scaled = act2 * 0.5  
    shifted = scaled + 0.1  
    final = relu(shifted)  
  
    return final
```

Your Task:

1. Identify fusion opportunities in this computation graph
2. Draw the optimized graph with fused operations
3. Estimate memory bandwidth reduction
4. Consider data layout optimizations



***Logits** are a model's raw, unnormalized scores output by the final layer before any activation like softmax or sigmoid.*

SESSION 2: MODEL CONVERSION AND COMPILATION

TensorFlow to TensorFlow Lite Conversion

Understanding the Conversion Pipeline

```
# Training model (TensorFlow)
def training_model(inputs):
    preprocessed = preprocess_images(inputs) # JPEG decode, resize
    features = feature_extractor(preprocessed)
    logits = classifier(features)
    predictions = softmax(logits)
    postprocessed = apply_nms(predictions) # Non-max suppression
    return postprocessed
```

Deployment Reality:

Most preprocessing and postprocessing operations aren't supported in TensorFlow Lite or are inefficient to implement as graph operations.

Solution Strategy:

Move non-core operations to **application code** (see next slide – application code vs. the model), export only the core neural network.



The application code vs. the model

Imagine you are building a mobile app that identifies plants from camera pictures.

The Model (TensorFlow Lite): This is the .tflite file you deployed

- A mathematical function that takes as input a tensor $[1, 224, 224, 3]$ of pixel values, and returns another tensor (e.g., a $[1, 1000]$ array of probabilities for 1000 plant classes).

The Application Code: This is everything else:

- The Java/Kotlin code for your Android app.
- The Swift/Objective-C code for your iOS app.
- The C++ code for an embedded device.
- The Python code for a backend server.

Operation Coverage Analysis

Understanding Supported Operations

TensorFlow Lite Operation Constraints:

```
# Check operation compatibility
def analyze_model_ops(model):
    supported_ops = get_tflite_supported_ops()
    model_ops = extract_ops_from_model(model)

    unsupported = []
    for op in model_ops:
        if op.type not in supported_ops:
            unsupported.append(op)

    return unsupported
```

LSTMs (Long Short-Term Memory networks) are a type of recurrent neural network (RNN) designed to handle long-term dependencies in sequential data. Unlike basic RNNs, which struggle to remember information over long sequences, LSTMs use special "gates" (forget, input, and output gates) to control what information is stored, updated, or discarded in their internal memory. This allows them to retain important context, while ignoring irrelevant details. LSTMs are useful for tasks like language translation, chatbots, and predicting sequences where past information is crucial for accurate results.

Common Unsupported Operations:

- LSTM and GRU layers (in microcontroller version – because of very limited RAM/Flash)
- Complex control flow operations
- String processing operations
- Advanced image processing operations

Gap Between Training and Deployment:

Transformation steps during export:

- Variables → Constants
- Training-only ops removed (backpropagation)
- Quantization transformations

GRUs (Gated Recurrent Units) are a simpler and more efficient version of LSTMs, designed to handle sequential data like text or time series. Unlike LSTMs, which have three gates (input, forget, output), GRUs use just two gates—an update gate (decides what past information to keep) and a reset gate (determines what to forget). They combine short-term and long-term memory into a single hidden state, making them faster to train while still capturing important patterns. GRUs are great for tasks like autocomplete, translation, or chatbots, where context matters but computational efficiency is key. They're newer (2014) and often perform as well as LSTMs with less complexity.

```

#include "tensorflow/lite/c/common.h"

typedef struct { float a, b; } OpData;

static void* Init(TfLiteContext* ctx, const char* buf, size_t len) {
  auto* d = (OpData*)ctx->AllocatePersistentBuffer(ctx, sizeof(OpData));
  d->a = 1.0f; d->b = 0.0f; // defaults; parse buf if you pass params
  return d;
}

static TfLiteStatus Prepare(TfLiteContext* ctx, TfLiteNode* node) {
  const TfLiteTensor* in = GetInput(ctx, node, 0);
  TfLiteTensor* out = GetOutput(ctx, node, 0);
  TF_LITE_ENSURE_EQ(ctx, in->type, kTfLiteFloat32);
  TF_LITE_ENSURE_STATUS(ctx->ResizeTensor(ctx, out,
    TfLiteIntArrayCopy(in->dims)));
  out->type = in->type;
  return kTfLiteOk;
}

static TfLiteStatus Eval(TfLiteContext* ctx, TfLiteNode* node) {
  const OpData* d = (const OpData*)node->user_data;
  const TfLiteTensor* in = GetInput(ctx, node, 0);
  TfLiteTensor* out = GetOutput(ctx, node, 0);
  const float *x = in->data.f; float *y = out->data.f; int n =
  NumElements(in);
  for (int i = 0; i < n; ++i) y[i] = d->a * x[i] + d->b;
  return kTfLiteOk;
}

TfLiteRegistration_V1* Register_MY_AFFINE() {
  static TfLiteRegistration_V1 r{Init, /*free=*/nullptr, Prepare, Eval,
    nullptr, kTfLiteCustom, "MY_AFFINE", 1};
  return &r;
}

// resolver.AddCustom("MY_AFFINE", Register_MY_AFFINE());

```

Custom Ops Implementation

When to use custom ops

- Only if no built-in op fits
- Expect larger binary + manual tuning

Lifecycle

- Register → Prepare (once) → Eval (each inference)

Prepare checklist

- Validate tensor count and types
- Set/resize output shape
- Allocate persistent data

Eval checklist

- Read inputs, fast loop over elements
- Avoid allocations; no logging in hot path

Portability and testing

- Guard platform code with ifdefs
- Unit test: shapes, types, edge cases
- Benchmark before/after

Automated Compilation Pipelines - XLA

```
import tensorflow as tf

# Simple model component
class Block(tf.Module):
    def __init__(self, c): super().__init__(); self.conv =
tf.keras.layers.Conv2D(c, 3, padding="same")
    def __call__(self, x, training=False):
        x = self.conv(x)
        x = tf.keras.layers.BatchNormalization()(x,
training=training)
        return tf.nn.relu(x)

# Enable XLA for this function (TF >= 2.11: jit_compile=True)
@tf.function(jit_compile=True)
def step(x):
    b = Block(32)
    return b(x, training=False)

# Warm up + run
x = tf.random.normal([1, 224, 224, 3])
y = step(x) # XLA compiles on first call; later calls use
optimized code
```

What XLA (Accelerated Linear Algebra) does

- Fuses ops into single kernels
- Picks efficient layouts (e.g., NHWC $\leftrightarrow$ NCHW)
- Reuses buffers to cut memory

When to use

- TensorFlow models with stable shapes
- Training or inference on CPU/GPU/TPU

When to be careful

- Highly dynamic Python control flow
- Ragged tensors or frequent shape changes

How to enable

- `@tf.function(jit_compile=True)` in TF 2.11+ (or `experimental_compile=True` in older TF)
- Or set `TF_XLA_FLAGS / tf.config.optimizer.set_jit(True)`

Automated Compilation Pipelines - TVM

What TVM does

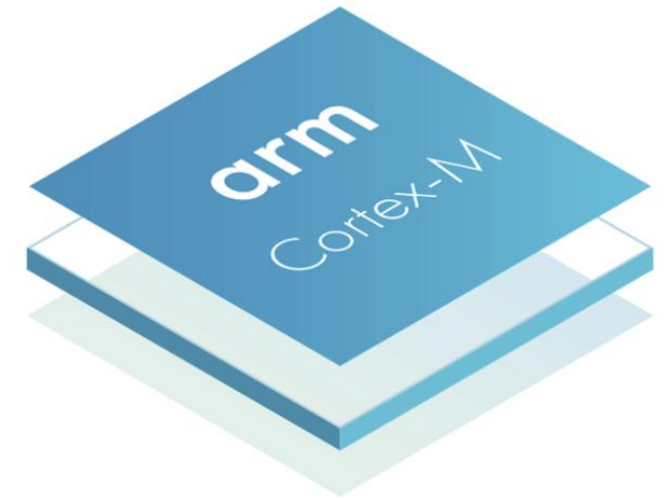
- Cross-framework import → auto-schedule/auto-tune → target-specific codegen
- Generates kernels for CPUs, GPUs, and MCUs

Why it's different from XLA

- Hardware-agnostic stack with search-based optimization
- Produces standalone artifacts (C/C++/LLVM) for deployment

When to use

- Heterogeneous fleets or non-TF models
- Squeezing perf on edge devices (ARM NEON, DSP, Tensor Cores)



ARM Cortex-M processors are a family of 32-bit RISC (Reduced Instruction Set Computer) microcontroller cores designed by Arm Limited. They are widely used in embedded systems, particularly for low-cost and energy-efficient applications.

Image source: community.arm.com

```
import torch, torch.nn as nn, tvm
from tvm import relay

m = nn.Sequential(nn.Conv2d(3,8,3,padding=1), nn.ReLU(), nn.AvgPool2d(2)).eval()
ex = torch.randn(1,3,64,64); ts = torch.jit.trace(m, ex).eval()
mod, params = relay.frontend.from_pytorch(ts, [("input0", ex.shape)])

target = tvm.target.Target("c -mcpu=cortex-m4") # emit portable C for Cortex-M4
with tvm.transform.PassContext(opt_level=3):
    lib = relay.build(mod, target=target, params=params)

open("model.c", "w").write(lib.get_source()) # compile with your MCU toolchain
```

Automated Compilation Pipelines - MLIR

3. Multi-Level Intermediate Representation (MLIR):

MLIR is **not a compiler itself** but a framework for building compilers. It enables **multi-stage optimizations** across different abstraction levels.

Key Features:

- **Flexible IR:** Can represent high-level (e.g., TensorFlow ops) and low-level (e.g., LLVM IR) code.
- **Domain-Specific Optimizations:** Supports dialects (e.g., `linalg` for linear algebra, `gpu` for GPU ops).
- **Reusable Passes:** Compiler transformations can be shared across frameworks.

How It Works:

A model is progressively lowered from high-level ops to hardware-specific instructions.

Example flow:

TensorFlow Graph → **MLIR (`tf dialect`)** → **MLIR (`linalg dialect`)** → **LLVM IR** → **Machine Code**

- `tf dialect` = TensorFlow-specific ops.
- `linalg dialect` = Generalized linear algebra ops for compiler-friendly optimizations.

Use Case:

Used internally in TensorFlow, PyTorch, and TVM for **flexible compiler pipelines**.



Hardware-Specific Optimizations - TPU

TPU Optimizations

Objective: Maximize throughput on Google's TPUs, which are specialized for matrix operations.

Key Considerations:

- TPUs perform best when matrix dimensions are aligned to specific sizes (e.g., multiples of 128).
- Misaligned dimensions can lead to underutilized hardware or wasted computation.

```
# Ensure matrix dimensions are multiples of 128
def tpu_optimize(model):
    for layer in model.layers:
        if isinstance(layer, Dense):
            # Pad dimensions for optimal TPU utilization
            layer.units = align_to_tpu_constraints(layer.units)
```

- **align_to_tpu_constraints:** A helper function that rounds up the number of units (neurons) in a dense layer to the nearest multiple of 128.
- **Why?** TPUs process matrix operations in tiles (e.g., 128x128 blocks). Aligning dimensions ensures full utilization of these tiles, avoiding inefficient partial computations.

Hardware-Specific Optimizations - GPU

GPU Optimizations

Objective: Optimize for parallel execution and memory bandwidth efficiency.

Key Considerations:

- GPUs excel at parallel workloads but require **coalesced memory access** (adjacent threads accessing contiguous memory).
- **Tensor Cores** (in modern GPUs) accelerate mixed-precision (FP16/FP32) operations.

Coalesced Memory Access:

- Ensure tensors are stored in memory layouts (e.g., NHWC for images) that allow threads to read contiguous blocks.
- Avoid strides or padding that disrupt memory locality.

Tensor Cores:

- Use mixed precision (FP16 for weights, FP32 for gradients) to leverage hardware acceleration.
- Enabled via frameworks like TensorFlow (`tf.keras.mixed_precision`).

Example: For a convolutional layer, prefer **channels last** (NHWC) over **channels_first** (NCHW) on NVIDIA GPUs (example in next slide).

Hardware-Specific Optimizations – GPU (cont)

GPU Optimizations

Example: For a convolutional layer, prefer **channels_last (NHWC)** over **channels_first (NCHW)** on NVIDIA GPUs.

In a Tensor with Batch size (N) = 1 (single image), Height (H) = 2, Width (W) = 2, Channels (C) = 3 (RGB)

Then the data for one image (2x2 RGB) is:

Pixel (0,0): [R00, G00, B00]

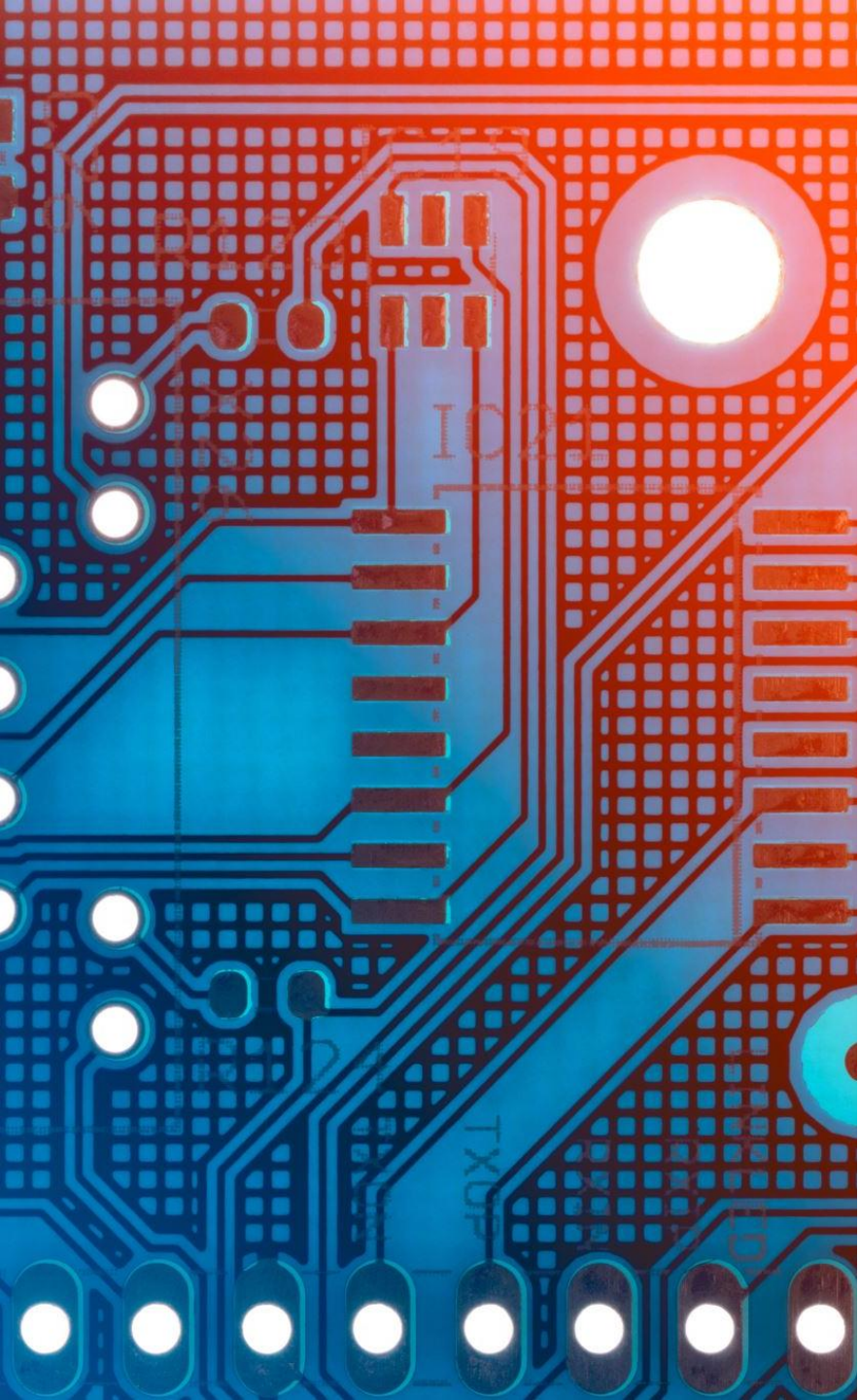
Pixel (0,1): [R01, G01, B01]

Pixel (1,0): [R10, G10, B10]

Pixel (1,1): [R11, G11, B11]

For a 2D convolution:

- In NHWC, threads processing a 3x3 patch load contiguous pixels (good locality).
- In NCHW, threads fetching the same patch scatter across memory (poor locality) – because channels are stored separately ([all R][all G][all B]).



Hardware-Specific Optimizations - Microcontrollers

Microcontroller Optimizations

Objective: Minimize memory and compute usage for resource-constrained devices (e.g., [ARM Cortex-M](#)).

Key Considerations:

- Limited RAM/Flash memory (e.g., KBs of memory).
- No OS or dynamic memory allocators; avoid heap usage.
- Cache misses are costly; optimize access patterns.

Small Data Types:

- Use 8-bit integers (int8) or binary weights instead of FP32 to save memory.
- Quantization is critical (e.g., TensorFlow Lite for Microcontrollers).

Cache-Friendly Access:

- Use linear or sequential memory access to avoid cache thrashing.
- Avoid random jumps (e.g., linked lists).

Static Memory Allocation:

- Pre-allocate buffers during compilation (no malloc at runtime).

Example:

- Replace a float32 dense layer with a int8 quantized layer, reducing memory by 4x.

Cross-Platform Compilation Challenges

- **Ops differ by target** -> Check early; replace missing ops (e.g., LSTM on MCU).
- **Memory budgets vary** -> Cloud: large; Mobile: limited; MCU: tiny.
- **Optimize per target** : Same model → different builds and passes.

Decision checklist

- Capability check
- Precision: FP16/TF32 (cloud) | INT8 (mobile) | INT8/INT4 (MCU)
- Graph: fuse, strip, fix static shapes
- Validate on device: accuracy, latency, memory fit

Target profiles

- Cloud: throughput, big batches, FP16/TF32
- Mobile: latency/battery, INT8 or mixed precision
- MCU: footprint first, INT8/INT4, static buffers

Example: One model → FP16 cloud, INT8 mobile, INT8 MCU (no LSTM)

Exercise 2 (Quiz)

Model Conversion Planning

Scenario: Speech Command Classifier

This function takes an audio file, processes it into a spectrogram, feeds it through a Convolutional Neural Network (CNN), and returns the model's best guess at what the sound is and whether it's confident enough in that guess.



```
def speech_classifier(audio_input):  
    # Preprocessing  
    decoded_audio = decode_wav(audio_input)  
    spectrogram = compute_fft(decoded_audio)  
    normalized_spec = normalize(spectrogram)  
  
    # Core neural network  
    conv1 = conv2d(normalized_spec,  
                    weights1)  
    relu1 = relu(conv1)  
    conv2 = conv2d(relu1, weights2)  
    relu2 = relu(conv2)  
    flattened = flatten(relu2)  
    logits = dense(flattened, dense_weights)  
  
    # Postprocessing  
    probabilities = softmax(logits)  
    top_prediction = argmax(probabilities)  
    confidence_threshold =  
        apply_threshold(probabilities, 0.8)  
  
    return top_prediction,  
        confidence_threshold
```

Exercise 2 (Quiz) – Question 1

In the original training function, which steps are clearly preprocessing?

- A. conv1, relu1
- B. decode_wav, compute_fft, normalize
- C. dense, softmax
- D. argmax



```
def speech_classifier(audio_input):  
    # Preprocessing  
    decoded_audio = decode_wav(audio_input)  
    spectrogram = compute_fft(decoded_audio)  
    normalized_spec = normalize(spectrogram)  
  
    # Core neural network  
    conv1 = conv2d(normalized_spec,  
                    weights1)  
    relu1 = relu(conv1)  
    conv2 = conv2d(relu1, weights2)  
    relu2 = relu(conv2)  
    flattened = flatten(relu2)  
    logits = dense(flattened, dense_weights)  
  
    # Postprocessing  
    probabilities = softmax(logits)  
    top_prediction = argmax(probabilities)  
    confidence_threshold =  
        apply_threshold(probabilities, 0.8)  
  
    return top_prediction,  
        confidence_threshold
```

Exercise 2 (Quiz) – Question 2

For deployment to a lightweight runtime, which section is most appropriate to package as the exported TFLite core?

- A. Only Preprocessing
- B. Preprocessing + Postprocessing
- C. Core neural network (conv/relu/flatten/dense)
- D. Only Postprocessing



```
def speech_classifier(audio_input):  
    # Preprocessing  
    decoded_audio = decode_wav(audio_input)  
    spectrogram = compute_fft(decoded_audio)  
    normalized_spec = normalize(spectrogram)  
  
    # Core neural network  
    conv1 = conv2d(normalized_spec,  
                    weights1)  
    relu1 = relu(conv1)  
    conv2 = conv2d(relu1, weights2)  
    relu2 = relu(conv2)  
    flattened = flatten(relu2)  
    logits = dense(flattened, dense_weights)  
  
    # Postprocessing  
    probabilities = softmax(logits)  
    top_prediction = argmax(probabilities)  
    confidence_threshold =  
        apply_threshold(probabilities, 0.8)  
  
    return top_prediction,  
        confidence_threshold
```

Exercise 2 (Quiz) – Question 3

If you were minimizing unsupported ops in the exported model, which of these would you most likely keep outside the model graph?

- A. conv2d layers
- B. ReLU activations
- C. FFT and normalization
- D. Dense layer producing logits



```
def speech_classifier(audio_input):  
    # Preprocessing  
    decoded_audio = decode_wav(audio_input)  
    spectrogram = compute_fft(decoded_audio)  
    normalized_spec = normalize(spectrogram)  
  
    # Core neural network  
    conv1 = conv2d(normalized_spec,  
                   weights1)  
    relu1 = relu(conv1)  
    conv2 = conv2d(relu1, weights2)  
    relu2 = relu(conv2)  
    flattened = flatten(relu2)  
    logits = dense(flattened, dense_weights)  
  
    # Postprocessing  
    probabilities = softmax(logits)  
    top_prediction = argmax(probabilities)  
    confidence_threshold =  
        apply_threshold(probabilities, 0.8)  
  
    return top_prediction,  
           confidence_threshold
```

Exercise 2 (Quiz) – Question 4

The core network ends with logits.
For a mobile app that needs just the top class, which postprocessing line would you implement in app code to decide the label?

- A. relu1
- B. argmax(probabilities)
- C. conv2
- D. flatten(relu2)



```
def speech_classifier(audio_input):  
    # Preprocessing  
    decoded_audio = decode_wav(audio_input)  
    spectrogram = compute_fft(decoded_audio)  
    normalized_spec = normalize(spectrogram)  
  
    # Core neural network  
    conv1 = conv2d(normalized_spec,  
                    weights1)  
    relu1 = relu(conv1)  
    conv2 = conv2d(relu1, weights2)  
    relu2 = relu(conv2)  
    flattened = flatten(relu2)  
    logits = dense(flattened, dense_weights)  
  
    # Postprocessing  
    probabilities = softmax(logits)  
    top_prediction = argmax(probabilities)  
    confidence_threshold =  
        apply_threshold(probabilities, 0.8)  
  
    return top_prediction,  
        confidence_threshold
```

Exercise 2 (Quiz) – Question 5

Suppose you want to reduce the exported model size without losing prediction quality. Which change aligns with the exercise structure?

- A. Remove conv layers
- B. Export logits and move softmax to the app
- C. Replace relu with tanh in the model
- D. Return probabilities instead of logits



```
def speech_classifier(audio_input):  
    # Preprocessing  
    decoded_audio = decode_wav(audio_input)  
    spectrogram = compute_fft(decoded_audio)  
    normalized_spec = normalize(spectrogram)  
  
    # Core neural network  
    conv1 = conv2d(normalized_spec,  
                    weights1)  
    relu1 = relu(conv1)  
    conv2 = conv2d(relu1, weights2)  
    relu2 = relu(conv2)  
    flattened = flatten(relu2)  
    logits = dense(flattened, dense_weights)  
  
    # Postprocessing  
    probabilities = softmax(logits)  
    top_prediction = argmax(probabilities)  
    confidence_threshold =  
        apply_threshold(probabilities, 0.8)  
  
    return top_prediction,  
        confidence_threshold
```

SESSION 3: LATENCY OPTIMIZATION STRATEGIES

The ultimate goal: making our models fast enough for real-world deployment.



Performance Profiling Fundamentals

Identifying Bottlenecks

1. Why Profile First?

Before optimizing, we must measure to identify the real bottlenecks.

Key Idea

By removing the ML inference call `tf.lite_interpreter.invoke()` and measuring the remaining time, we determine:

- **Non-ML latency** (e.g., data preprocessing, post-processing, I/O).
- If total time **does not decrease significantly**, the bottleneck is **not in ML inference** but elsewhere (e.g., disk reads, network calls).
- If time drops drastically, **ML inference is the bottleneck**, and optimizations should focus there.

Why This Matters:

- Avoids premature optimization—fixing parts that don't impact overall performance.
- Ensures effort is spent where it matters most.



Time breakdown for bird detection system (example)

- Image capture: 1.5 seconds
- ML inference: 0.3 seconds
- Result processing: 0.2 seconds

Performance Profiling Fundamentals

2. Profiling Hierarchy

A structured approach to drill down into performance issues:

A. System-level

- Measures **total application latency** (end-to-end time).
- Example: How long does the entire bird detection system take?

B. Framework-level

- Focuses on **ML inference time** (e.g., TensorFlow Lite, PyTorch runtime).
- Example: Is the model itself slow, or is it the framework overhead?

C. Operation-level

- Breaks down **individual layer timings** (e.g., convolutions, matrix multiplications).
- Example: Is one particular layer in the neural network causing delays?

D. Hardware-level

- Checks **memory bandwidth, compute utilization** (CPU/GPU/TPU).
- Example: Is the system memory-bound or compute-bound?

Model Architecture Optimization

1. FLOP Estimation

Dense (Fully Connected)

- $\text{FLOPs} \approx 2 \times (\text{input_size} \times \text{output_size})$
- Example: $1024 \rightarrow 512 \Rightarrow 2 \times 1024 \times 512 = 1.05\text{M FLOPs}$

Convolution (standard, no bias)

- $\text{FLOPs} \approx 2 \times (k_h \times k_w \times \text{in_ch}) \times (\text{out_h} \times \text{out_w}) \times \text{out_ch}$
- Example: 3×3 , $\text{in_ch}=64$, $\text{out_ch}=128$, $\text{output}=56 \times 56$
 - $2 \times 3 \times 3 \times 64 \times 56 \times 56 \times 128 \approx 11.1\text{G FLOPs}$

Notes

- “2×” counts multiply plus add.
- Grouped/depthwise: divide by groups; depthwise $\Rightarrow$ groups=in_ch, out_ch=in_ch.
- FLOPs help rank hotspots; also consider memory and bandwidth.

Model Architecture Optimization

2. Architecture Comparison

Model A: 100M FLOPs → 1000ms on target hardware

Model B: 50M FLOPs → expect ~500ms

Model C: 200M FLOPs → expect ~2000ms

Key Insight: FLOPs scale **linearly with latency** (assuming similar hardware utilization).

- Halving FLOPs (Model B) ≈ halves latency.
- Doubling FLOPs (Model C) ≈ doubles latency.

Purpose: Compare models by **FLOPs vs. accuracy**.

- Loads candidate models (e.g., different CNN architectures).
- Computes total FLOPs and evaluates accuracy.
- Outputs a trade-off analysis (e.g., "Is 2× FLOPs worth +1% accuracy?").

Model Architecture Optimization

3. Optimization Strategies

1. Remove Layers with Highest FLOP/Accuracy Ratio

Example: Removing a 1M-FLOP dense layer that improves accuracy by only 0.1%.

2. Replace Expensive Operations with Efficient Alternatives

- **Examples:**

- Replace 3×3 convolutions with 1×1 convolutions (9× fewer FLOPs).
- Use **grouped convolutions** (e.g., ResNeXt) to reduce connectivity.

3. Use Depthwise Separable Convolutions

Splits a standard convolution into:

- **Depthwise Convolution:** Filters per input channel ($\text{kernel_h} \times \text{kernel_w} \times \text{input_channels}$).
- **Pointwise Convolution:** 1×1 conv to mix channels ($\text{input_channels} \times \text{output_channels}$).

FLOP Reduction:

- Standard conv: $K \times K \times C_{in} \times H \times W \times C_{out} \times 2$.
- Depthwise separable: $(K \times K \times C_{in} \times H \times W \times 1 \times 2) + (1 \times 1 \times C_{in} \times H \times W \times C_{out} \times 2)$.
- Example: For $C_{in}=32$, $C_{out}=64$, $K=3$, this reduces FLOPs by ~8–9×.

Quantization for Latency

Core idea

- Reduce numerical precision of weights and activations to shrink models and speed up inference, especially on low-precision hardware.

Two approaches

- Post-training weight quantization (weights only)
 - Convert FP32 weights to INT8/FP16.
 - ~75% size reduction moving FP32 $\rightarrow$ INT8 (4:1).
 - Faster loads, lower bandwidth, better cache fit.
 - Limitation: activations stay FP32, so speedups are partial.
- Post-training integer quantization (weights + activations)
 - Convert both to integers; uses integer compute units.
 - Requires a representative dataset to calibrate activation ranges.
 - Big gains on integer-optimized hardware; must validate accuracy carefully.

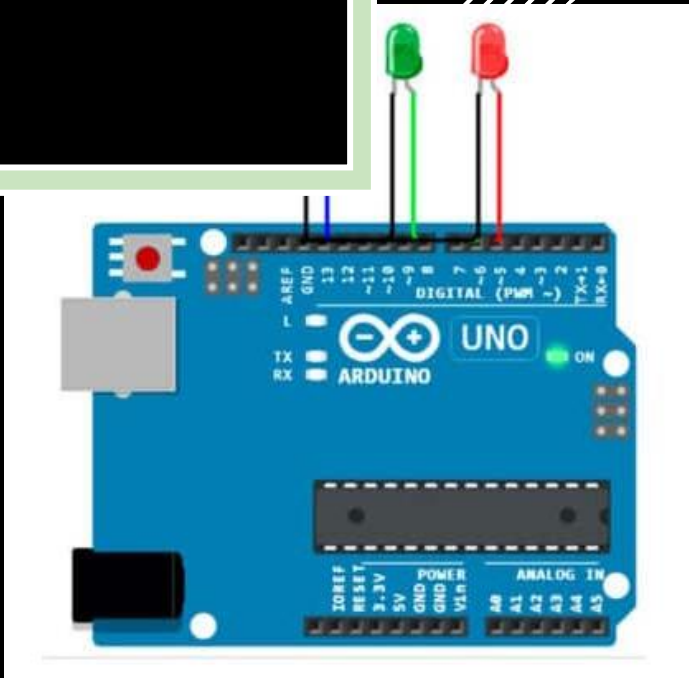
Hardware angle

- Microcontrollers: INT8 MACs are far faster; often no FP unit $\rightarrow$ integer quantization is essential.
- Mobile NPUs/DSPs: designed for low-precision arithmetic (e.g., Hexagon, Neural Engine).
- Universal wins: lower memory bandwidth, better cache locality, reduced power.



Code-Level Optimizations

GPIO - General Purpose Input/Output, in Arduino refers to the microcontroller's pins that can be configured as either inputs to read data from sensors or buttons, or as outputs to control devices like LEDs or motors.



LED Timing (hardware-accurate)

How to

- Toggle a GPIO before and after the code;
- measure pulse width on scope/LA.

Why it's great

- Near-zero overhead, hardware-accurate, immune to software/print/interrupt noise.

When to use

- Real-time loops; tight budgets (e.g., <10 ms); sanity-check true wall time.

Caveats

- Needs external tool; gives total time only, not per-operation breakdown.

Code-Level Optimizations



Timer-Based Profiling (convenient logging)

How to

- Use `micros()`/HAL timers to record start/end and log results.

Pros

- No external gear; easy to automate and collect over many runs.

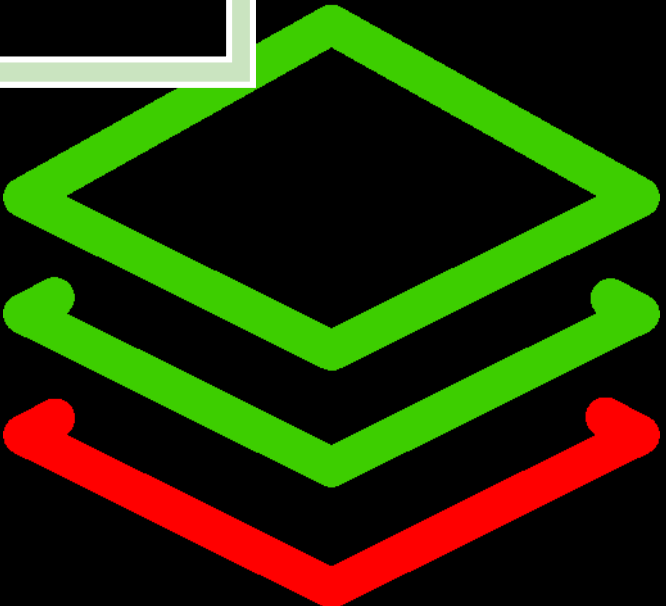
Cons

- Overhead from timer calls and serial I/O can skew results.

Tips

- Buffer timings in RAM; dump after the critical section completes.
- Disable prints inside tight loops; measure overhead once and subtract.

Code-Level Optimizations



Operation-Level Profiling (find bottlenecks)

How to

- Measure time per op/layer to see where latency concentrates.

Value

- Pinpoints hot spots (e.g., conv, GEMM, activations) to target optimizations.

Practical notes

- MCU ML stacks may lack per-op hooks—may need to add timing around ops.
- Sampling a subset of ops reduces overhead; profile representative inputs.

Hardware-Specific Optimizations

```
// Use platform-specific optimized kernels
// Instead of portable reference implementations
make -f tensorflow/lite/micro/tools/make/Makefile \
    TARGET=cortex_m_generic \
    TAGS="cmsis-nn" // Use ARM CMSIS-NN optimizations
```

<https://github.com/ARM-software/CMSIS-NN/releases>

1. Optimized Implementations (Platform-Specific Kernels)

Instead of using generic, portable implementations, optimized kernels tailored for specific hardware (e.g., ARM Cortex-M) can significantly improve performance.

Example: Using ARM CMSIS-NN for TensorFlow Lite Micro. CMSIS-NN is a collection of highly optimized neural network kernels for ARM Cortex-M processors.

Why?

- Reference implementations (portable but unoptimized) may run slowly on microcontrollers.
- CMSIS-NN provides hand-tuned assembly/C kernels for operations like convolution, matrix multiplication, and activation functions.
- Can provide **2-5x speedup** over naive implementations.

Benefits:

- **Reduced latency** (faster inference).
- **Lower power consumption** (critical for battery-powered devices).
- **Better utilization of DSP/SIMD instructions** (e.g., ARM NEON, Helium).



Hardware-Specific Optimizations

```
// Example: Vectorized operations on ARM Cortex-M
void optimized_conv2d_kernel(const int8_t* input,
                             const int8_t* weights,
                             int32_t* output) {
    // Use ARM NEON or Helium instructions
    // 4-8x speedup for appropriate operations
}
```

2. SIMD Instruction Utilization:

Key Idea:

- Single Instruction Multiple Data (SIMD) allows processing multiple data points in parallel, drastically improving throughput.
- Example: ARM NEON/Helium for Convolution

NEON (ARM Cortex-A) and Helium (ARM Cortex-M55/M85) are SIMD extensions.

How it works

- Instead of processing one multiply-accumulate (MAC) at a time, SIMD processes 4–16 operations in a single cycle.
- Example: A 128-bit NEON register can hold four 32-bit floats or sixteen 8-bit integers.

Why?

- Convolutions and matrix multiplications are compute-bound; SIMD reduces instruction count.
- Can achieve 4–8x speedup for 8-bit quantized models.

When to Use SIMD:

- Data-parallel workloads (e.g., image processing, ML inference).
- Low-precision arithmetic (int8, int16) benefits most.

Hardware-Specific Optimizations

```
// Optimize for cache-friendly access patterns
void tiled_matrix_multiply(const float* a, const float* b, float* c) {
    // Process in cache-sized tiles
    const int TILE_SIZE = 32; // Fits in L1 cache
    for (int i = 0; i < n; i += TILE_SIZE) {
        for (int j = 0; j < m; j += TILE_SIZE) {
            multiply_tile(&a[i*k], &b[j], &c[i*m + j]);
        }
    }
}
```

3. Memory Access Optimization (Cache-Friendly Patterns)

Key Idea:

- Modern CPUs have cache hierarchies (L1, L2, L3). Optimizing memory access patterns reduces cache misses and improves performance.

Problem:

- Naive matrix multiplication has poor locality (strided memory access → cache thrashing).
- Example: For large matrices, accessing $b[j]$ may evict useful cache lines.

Solution: Tiling (Blocking)

- Break computation into smaller tiles that fit in L1/L2 cache.
- Each tile is reused multiple times before eviction

Why Tiling?

- Reduces DRAM bandwidth pressure.
- Can provide 2–3x speedup for large matrices.

When to Use Tiling:

- Large matrix multiplications (common in deep learning).
- Any memory-bound operation (e.g., image processing, FFTs).

Product Design for Latency

1. Hybrid Processing Approaches

- Combine fast approximate with slow accurate to cut average latency.
- Example: Optical flow tracking in ~ 5 ms; ML classifier ~ 100 ms.
- Strategy: Run ML only when the fast path is uncertain or at intervals.
- Result: 90% fast path, 10% slow path $\rightarrow$ average ≈ 15 ms ($\sim 7\times$ faster).
- Note: Engineer smooth handoffs and drift correction between paths.



<https://codelabs.developers.google.com/magicwand#0>

Product Design for Latency

2. Hierarchical Processing

- Multi-stage filters reserve heavy compute for ambiguous inputs.
- Typical behavior: <2% reach Stage 3 → $\approx 50\times$ compute reduction.
- Predictable worst case: 100 ms only when truly needed.

Three-Stage Wake Word Detection:

Stage 1 (Energy Check):

- Rejects 50%+ of audio frames (silence/noise).
- Simple DSP (Digital Signal Processing) operation (e.g., RMS (Root Mean Square) calculation).

Stage 2 (Quick Match):

- Uses a small model (e.g., DS-CNN or MFCC-based HMM).
- Filters 90% of non-wake-word speech.

Stage 3 (Full Model):

- Runs only on 1–2% of inputs that pass earlier stages.

Product Design for Latency



Listening

3. User Experience Design

Mask latency through UI/UX techniques that manage user expectations

- **Uncertainty visualization:** show confidence (e.g., semi-transparent gesture trail) so users adjust behavior.
- **Progressive disclosure:** immediate status, then partial results, then final.
- **Latency-tolerant interactions:** allow queuing; predictive previews for likely outcomes.

Payoff

- 2–5× drop in perceived latency without changing raw performance.
- Often faster to ship and improves satisfaction immediately.

Memory Bandwidth Optimization

1. Memory Access Patterns

Core message

CPUs compute faster than they can fetch memory; prefetchers love sequential access.

Poor: outer loop over channels → memory jumps → cache misses → stalls.

Better: channels in the inner loop → adjacent addresses → effective prefetch → high utilization.

Impact

Can raise CPU utilization from ~20% to ~80% in image ops—about 4× faster.

Tip

Align loops with layout (e.g., NHWC vs NCHW). Keep the innermost loop traversing contiguous memory.

Memory Bandwidth Optimization

2. In-place Operations:

Core message

Avoid unnecessary copies; they double memory traffic and pressure.

Many ops (ReLU, clamp, simple norms, elementwise arithmetic) can overwrite inputs safely.

Benefits

Halves bandwidth for the op; reduces peak memory footprint.

Especially valuable on MCUs and also pays off on desktops by reducing DRAM traffic.

Safeguards

Only do in-place when no later step needs the original tensor.

Watch for aliasing with views/slices.

Memory Bandwidth Optimization

3. Memory Pool Management:

Core message

Dynamic allocation (malloc/new) adds overhead and fragmentation—bad for real time.

Preallocate a pool/arena and sub-allocate from it for tensors/workspaces.

Benefits

Predictable latency, fewer stalls, stable memory usage.

Matches how TF/PyTorch allocators and many embedded runtimes operate.

Embedded note

Only do in-place when no later step needs the original tensor.



Benchmarking and Validation - Comprehensive Benchmarking

- Measure all three: accuracy, latency (cold and warm), and peak memory.
- Optimizations are trade-offs; define acceptable accuracy loss for your app.

Accuracy validation

- Prevent functional regressions; a 10× speedup isn't useful if accuracy drops 50%.
- A 2× speedup with ~1% loss may be a win—depends on tolerance and domain.

Latency measurement

- Capture cold-start vs warm throughput; control for cache/heating/jitter.
- Use medians with percentiles (p50/p95/p99) over multiple runs.



Benchmarking and Validation - Validation Across Conditions

Dimensions to cover

- Different input patterns: typical, edge, and adversarial; various sizes/resolutions.
- Hardware diversity: GPUs/CPUs/NPUs/MCUs; high-end vs budget; different vendors.
- System modes: single vs multi-threaded, power-save vs performance, throttled vs cool.

Why it matters

- Optimizations can be device- or data-shape-specific; portability must be proven.
- Avoid surprises where a fast server model becomes slow on-device.



Benchmarking and Validation - Edge Cases and Stress Testing

What to test

- Rare but critical inputs; boundary conditions (empty/malformed, max batch, long sequences).
- Load spikes: concurrency, rapid successive calls, long-duration soak tests.
- Environmental noise: background audio, accents; out-of-distribution gestures.

Outcomes

- Reveal instability, tail-latency blowups, memory leaks, or accuracy cliffs.
- Prevent the “works in the lab, fails in production” scenario with confidence gates.

Summary - Week 3 Key Takeaways

Graph-level Optimization Mastery

Core Optimization Techniques:

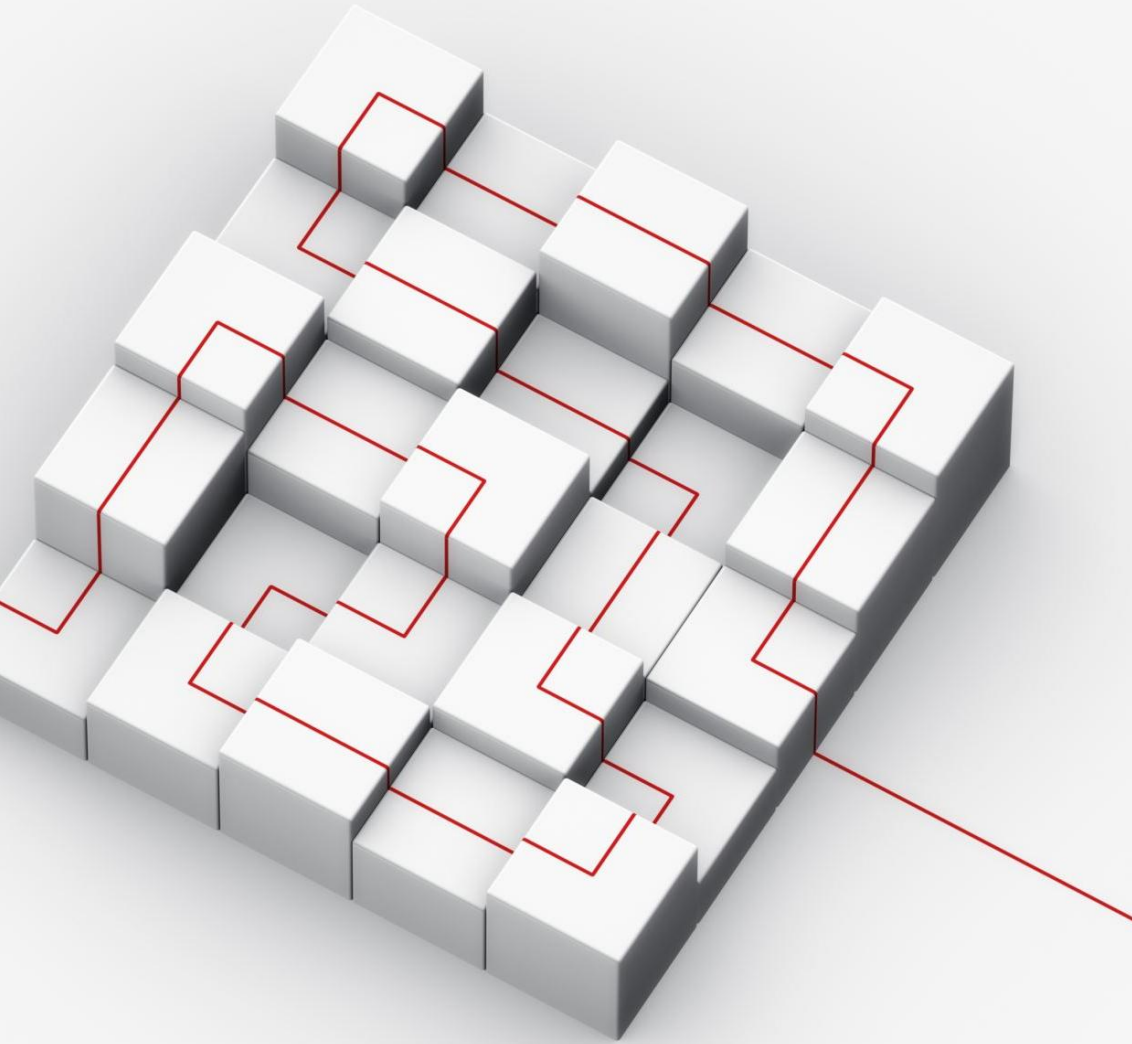
- **Graph transformations:** Kernel fusion, redundancy elimination
- **Memory optimization:** Access pattern improvement, bandwidth reduction
- **Model conversion:** Strategic separation of core ML from preprocessing
- **Latency optimization:** Multi-level approach from architecture to code

System Design Principles:

- **Profile first:** Identify actual bottlenecks before optimizing
- **Optimize holistically:** Consider entire system, not just ML model
- **Validate thoroughly:** Ensure optimizations don't hurt accuracy
- **Platform-aware:** Tailor optimizations to target hardware capabilities

Practical Impact:

- Enable real-time inference on resource-constrained devices
- Reduce energy consumption through efficiency improvements
- Make edge deployment practical for complex models
- Bridge the gap between training and deployment environments



Questions and Discussion

Today's Key Concepts:

- Computation graph optimization techniques
- Model conversion strategies for deployment
- Latency optimization across system levels
- Hardware-aware optimization approaches

Think about:

- When should you move operations out of the ML model into application code?
- How do you balance optimization aggressiveness with accuracy preservation?
- What are the key considerations for cross-platform deployment?

Looking Ahead: Week 4 - Parallelism in ML Systems

Next Week's Focus:

- Parallelism strategies from data center to microcontroller
- Energy-efficient parallel processing
- Distributed processing challenges
- Size constraints and parallel execution trade-offs

Required Readings for Week 4:

- Book 1 - Volume 2: Distributed Training
- Book 1 - Volume 2: Collective Communication
- Book 1 - Volume 2: Edge Intelligence
- Book 1 - Volume 1: Model Compression
- Book 2: Chapter 16: Optimizing Energy Usage
- Book 2: Chapter 17: Optimizing Model and Binary Size



Assignment 2 — Multi-Scale Optimization

Due: Week 5 — Sep 29, 23:59

Objective

- Optimize one CIFAR-10 model for cloud and edge; compare compute, memory, latency, accuracy.

Parts & Deliverables

1. Baseline (15 pts)

- Over-parameterized CNN + callbacks (ES/ReduceLROnPlateau/Checkpoint), >70% test acc.
- Submit: code, training curve, params/model size, baseline accuracy & latency.

2. Cloud Optimization (20 pts)

- Mixed precision; distributed training; large-batch throughput (grad accumulation + tf.data); knowledge distillation.
- Submit: implementations + benchmarks (speed, memory, accuracy, scaling).

3. Edge Optimization (20 pts)

- Pruning (~75% sparsity) + fine-tune; quantization (dynamic, INT8, FP16); lighter architecture; TFLite conversion.
- Submit: sparsity vs accuracy, quantization comparison, TFLite sizes/latency, edge viability.

4. Multi-Scale Pipeline (25 pts)

- Automated optimize() for targets: cloud_server, edge_device, microcontroller.
- Output: per-target results, Pareto trade-offs, recommendations, JSON report.

5. Analysis (20 pts)

- 3–4 pages: trade-off table (size/acc/latency/memory/power/complexity), effectiveness of cloud/edge strategies, deployment recs for:
 - A: Real-time video (<50 ms) → Edge + cloud backup
 - B: IoT (<1 mW) → TinyML + periodic cloud sync
 - C: Mobile (offline) → Multi-tier
- Include dev trade-offs, production extensions, future trends.

Submission

- Code: part1_baseline.py, part2_cloud_optimization.py, part3_edge_optimization.py, part4_deployment_pipeline.py, requirements.txt, README.md
- Models/Artifacts: baseline_model.keras, cloud_optimized_models/, edge_optimized_models/, *.tflite, multi_scale_optimization_report.json, multi_scale_analysis.pdf, charts
- Demos: demo_notebook.ipynb, inference + benchmarking scripts